

MIRACLE simulation outputs metadata specification version 3.0

Gary Polhill Lorenzo Milazzo Dawn Parker Xiongbing Jin
Calvin Pritchard Ju-Sung Lee Tatiana Filatova Alexey Voinov
Terry Dawson Doug Salt

29 April 2016

Abstract

This document contains metadata specifications for recording simulation outputs.

Contents

ChangeLog	6
Introduction	7
Overview	7
Analysis	9
External ontologies	9
Fine grain	11
Folksonomy	12
Project	12
Provenance	12
Services	15
Workflow	15
Detailed specifications	18
Attributes	18
Annotation	19
Standards	19
Automation	19
Attributes	19
Relationships	19
Application	19
Description	19
Standards	19
Automation	19
Attributes	20
Relationships	20
Argument	21
Description	21
Standards	21
Automation	21
Attributes	21
Relationships	22

ArgumentValue	23
Description	23
Standards	23
Automation	23
Attributes	23
Assumes	23
Standards	23
Automation	24
Relationships	24
Assumption	24
Description	24
Standards	24
Automation	24
Attributes	24
Box	25
Description	25
Standards	25
Automation	25
Attributes	25
Relationships	26
BoxType	27
Description	27
Standards	27
Automation	27
Attributes	27
Computer	27
Description	27
Standards	27
Automation	28
Attributes	28
Content	28
Description	28
Standards	28
Automation	28
Attributes	28
Relationships	29
Context	29
Description	29
Standards	29
Automation	30
Attributes	30
Relationships	30
Contributor	30
Description	30
Standards	30
Automation	30
Attributes	30

Relationships	31
Dependency	31
Description	31
Standards	31
Automation	31
Attributes	31
Relationships	32
Documentation	32
Description	32
Standards	32
Automation	32
Attributes	32
Relationships	32
Employs	33
Description	33
Standards	33
Automation	33
Relationships	33
Entailment	34
Description	34
Standards	34
Automation	34
Relationships	34
Implements	34
Description	34
Standards	34
Automation	35
Attributes	35
Relationships	35
Input	35
Description	35
Standards	35
Automation	36
Attributes	36
Relationships	36
Involvement	36
Description	36
Standards	36
Automation	36
Attributes	36
Relationships	37
Meets	37
Description	37
Standards	37
Automation	37
Relationships	37

Model	37
Description	37
Standards	38
Automation	38
Attributes	38
Relationships	38
Parameter	38
Description	38
Standards	38
Automation	38
Attributes	38
Relationships	39
Person	39
Description	39
Standards	39
Automation	39
Attributes	39
PersonalData	40
Description	40
Standards	40
Automation	40
Attributes	40
Relationships	40
Pipeline	40
Description	40
Standards	40
Automation	41
Attributes	41
Relationships	41
Process	41
Description	41
Standards	41
Automation	41
Attributes	41
Relationships	42
Product	42
Description	42
Standards	43
Automation	43
Attributes	43
Relationships	43
Project	43
Description	43
Standards	43
Automation	43
Attributes	44
Relationships	44

Requirement	44
Description	44
Standards	44
Automation	44
Relationships	44
Specification	45
Description	45
Standards	45
Automation	45
Attributes	45
Relationships	46
StatisticalInput	46
Description	46
Standards	46
Automation	46
Relationships	46
StatisticalMethod	46
Description	46
Standards	47
Automation	47
Attributes	47
StatisticalVariable	47
Description	47
Standards	47
Automation	47
Attributes	47
Relationships	47
Statistics	48
Description	48
Standards	48
Automation	48
Attributes	48
Relationships	48
Study	49
Description	49
Standards	49
Automation	49
Attributes	49
Relationships	49
Tag	49
Description	49
Standards	50
Automation	50
Attributes	50
TagMap	50
Description	50
Standards	50

Automation	50
Relationships	50
User	51
Description	51
Standards	51
Automation	52
Attributes	52
Relationships	52
Uses	52
Description	52
Standards	52
Automation	52
Attributes	52
Relationships	52
Value	53
Description	53
Standards	53
Automation	53
Attributes	53
Relationships	53
Variable	55
Description	55
Standards	55
Automation	55
Attributes	55
Visualisation	56
Description	56
Standards	56
Automation	56
Attributes	56
Relationships	56
VisualisationMethod	57
Description	57
Standards	57
Automation	57
Attributes	57
VisualisationValue	57
Description	57
Standards	57
Automation	57
Relationships	57
Bibliography	58
 ChangeLog	

Version	Date	Person	Description
1.1.3	28 April 2015	Lorenzo Milazzo and Gary Polhill	Original version
1.1.4	18 June 2015	Gary Polhill, Lorenzo Milazzo, Dawn Parker, Calvin Pritchard, Xiongbing Jin	Enhancements for workflow specifications and links with a metadata schema from earlier work at the University of Waterloo
1.1.5	7 July 2015	Gary Polhill and Lorenzo Milazzo	Improvements to fine grain, assumptions and statistics; explicit links to possible external ontologies; record of visualisations
1.1.6	26 Nov 2015	Ju-Sung Lee and Gary Polhill	Improvements to the fine grain; cosmetic improvements
1.1.7	29 Apr 2016	Doug Salt	Corrections. Add Description field to everything. Add About field to everything. Remove Purpose from Pipeline as redundant. Remove Purpose from Application as redundant. Change statisticalInput, statisticalVariable and statisticalMethod
2.0.0	8 Nov 2022	Doug Salt	Tidied up the relations
3.0.0	9 Sep 2026	Doug Salt	Converted to markdown and updated for RESAS release

Introduction

The replication ‘crisis’ (Fanelli 2018) in science generally (and especially the social sciences (Shrout and Rodgers 2018)) has its correlate in social simulation (Edmonds and Hales 2003). In our own work (Polhill et al. 2017), the complicated computational workflows associated with preparing, running and analysing the results of agent-based models have demonstrated the need for automated record-keeping.

Earlier work identified the various ways in which provenance metadata could be used to help analyse agent-based models (Pignotti et al. 2013). Three types of provenance were identified in that work:

1. The social processes outlining the history of model development.
2. Execution of the model and associated code for analysing its results.
3. Detailed state-change metadata per run.

This document provides a framework in which the first two of these types of provenance can be recorded.

Overview

This specification builds on earlier work towards metadata standards for sharing simulation outputs (Polhill et al. 2014). The schema diagram is shown in fig. 2.

Note originally this schema was designed to run on a relational database, but later work has adapted it to run on graph database as well, due to ease of querying on a graph database (Davoudian and Liu 2020). As such there are many reified relationships in the relational description of the database. Please note that in the graph database, the reified relationships have been converted to direct links between entities; the many to many relation being more natural in a graph database setting. A note has been made when a reified relationship in the relational schema has been replaced with a direct link in the graph database.

There are two important dimensions of distinction. First is fine- versus coarse- grained metadata. Second is provenance versus workflow. Coarse-grained metadata describes how particular files come (or came) into being, or were (or could be) used to bring other files into being. Fine-grained metadata describes specific values

recorded in social simulation outputs. To make the distinction concrete, suppose a simulation produces a CSV file. The data within the CSV file are covered by fine-grained metadata, whilst the fact that the simulation produces the CSV file is coarse-grained. Turning to the other dimension, provenance metadata describes what actually happens (run W of simulation X produced output file Y), whilst workflow metadata describes what could happen (simulation X produces an output file of type Z). The distinctions are summarised in fig. 1.

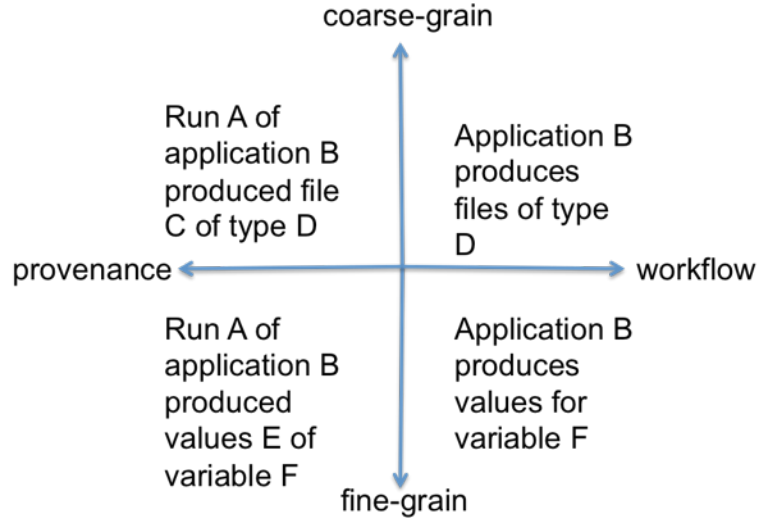


Figure 1: Dimensions of metadata

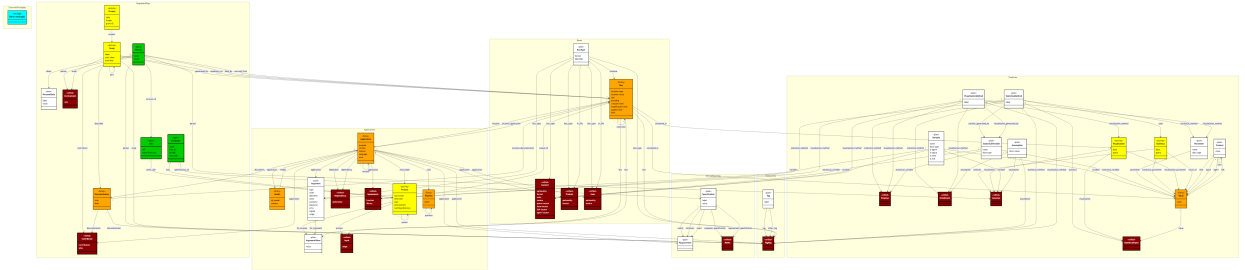


Figure 2: Metadata schema for MIRACLE. Boxes indicate tables, arcs represent relations, with 1 indicating a singular relations, “*” indicating ‘many’ part of a many-to-one or many-to-many relationship. Tables are coloured in brown if they are breaking a many-to-many relationship (a reified relationship), with yellow, orange and green being used to denote specialisations from the PROV standard Activity, Entity and Agent classes respectively. Blue boxes show external databases/ontologies to which this one could be linked.

With the schema now having a large number of tables, it is better to consider it in parts. The following subgraphs of the schema (which may partially intersect) cover different topic areas of the whole:

- **Analysis:** Part of the fine grain pertaining only to analysis and visualisation.
- **External:** Links to external ontologies.
- **Fine-grain:** All fine-grain metadata.
- **Folksonomy:** Tables allowing tagging.
- **Project:** Metadata about projects.
- **Prov:** Tables capturing provenance metadata.
- **Services:** Tables pertaining to service-provision and matching requirements against specifications.

- **Workflow:** Tables relating to workflow.

The overview section now continues with an explanation of each subgraph in turn, with consideration given to the degree of user interaction and maintenance. The remainder of the document explains each table in detail.

Analysis

The subgraph for recording metadata about Analysis is shown in fig. 3. A **Variable** is metadata about the contents of files, and may have more than one **Value**. A **Value** may be visualised, and **Statistics** may be computed using them. **Statistics** are computed using **StatisticalMethods**, and Visualisations constructed using **VisualisationMethods**. **StatisticalMethods** produce **StatisticalVariables** as outputs, which are stored in the **Values** table as results-of **Statistics**. **Statistics** and **Visualisations** are thus conceived as a **PROV::Activity** in the PROV vocabulary (Gao et al. 2020).

When a user of the system has identified some statistics or visualisations they want to be able to reuse in other studies or record provenance about, as part of recording the activity, if the statistics are not something already added by another user, they would make an entry in the **StatisticalMethods:** or **VisualisationMethods** tables. If known, the user would also record any **Assumptions Entailment** by the methods. This information is not a requirement, as the user may not have relevant expertise to assert that a particular computation involves an **Assumption**; however, this information can be added at any time by any user. The **Employs** table can also be filled with data recording when one **StatisticalMethod** or **VisualisationMethod** uses a **StatisticalVariable** as input.

Assumptions could be quite trivial – in the most simple case, for example, that a numeric **Variable** is assumed to be a cardinal (as opposed to ordinal or nominal) when computing the mean of its **Values**. For other statistics, **Assumptions** can record such things as whether the **Variable** is normally distributed, or has constant variance.

Once an **Assumption** has been stated as **Entailment** by a **StatisticalMethod** or **VisualisationMethod**, it can be automatically inferred that **Persons** have made **Assumptions** about **Variables**, where they have done **Statistics** or **Visualisations** that use the **Methods** and **Values** of those **Variables** that are returned by the queries used to get the raw data on which the activities operated. The query is stored, along with the date made, in order to avoid populating large relational tables recording the **Visualisations** and **Statistics** that used **Values** as input data. Note that **Values** may also be used to store **Parameters** and **StatisticalVariables** that act to configure **Visualisations** and **Statistics**. These are captured using the visualisation-parameter and statistical-parameter relations, and the **StatisticalInput** table.

Visualisations are automatically inferred when the **Box** they are contained-in has a **BoxType** with a **VisualisationMethod** in it. The **Values** visualised (recorded in the **VisualisationValue** table) can be (possibly somewhat dubiously) inferred from the **Input** to the **Application** that ran the **Process** that created the **Box**, until such time as statistical tools support logging this information in sufficient detail.

External ontologies

Though not explicitly noted in the tables, with the exception of OpenABM (with which this schema had to integrate for the MIRACLE project), various external ontologies and schemas are relevant, and could be linked to if required. There are various ways such links could be manifested (relational tables for example, implementing each of the blue dashed lines in fig. 4), and those suggested are by no means exhaustive. Neither is it necessarily the case that any specific ontology or external schema is proposed or adopted. The following gives examples of specific external links:

- **Bib** – connection to a bibliographic database (e.g. bibsonomy). Note that the **Documentation** table is intended to be quite generic, and include journal and conference articles as well as reports and code documentation.
- **Geo** – links to ontologies or databases containing geographical or spatial concepts, such as GeoSparql and WGS84. The idea here is that we may want to link various table entries to external geographical databases, for example, to say that a **Study** pertained to a particular region, or that a **Box** is a GIS file.

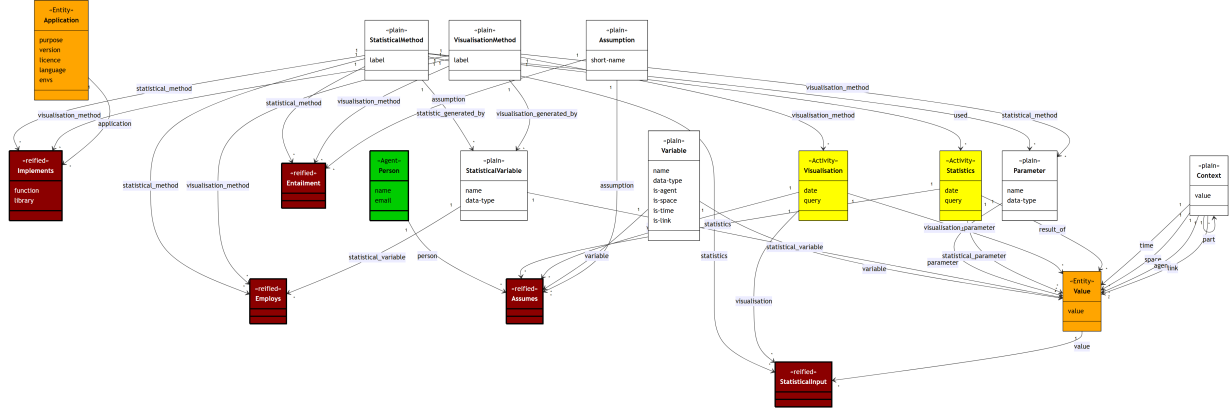


Figure 3: The analysis subgraph of the schema

- OpenABM – links to the CoMSES-Net archive of agent-based models, and is essential in order to identify which model these metadata are describing the output analysis of. (Although in principle, the system described could be applied to any stage of modelling, including setting up the model files, running the model and analysing the output, the focus of the MIRACLE project is specifically on the output analysis.)
- Services – links to vocabularies describing the requirements of applications and capabilities of service-providers. Where WSDL and OWL-S were once the natural reference points here, most service description in practice has since moved to REST-style APIs described using OpenAPI (formerly Swagger), which is now the de facto standard for machine-readable service interfaces. OpenAPI lacks OWL-S’s semantic-reasoning ambitions, but its wide tooling support and adoption make it a more practical link for automatic discovery of applications and services, including their input and output specifications.
- SocialWeb – we may want to allow people to link to social web tools such as ResearchGate, LinkedIn, Facebook and Twitter. The FOAF ontology also has attributes that we can draw on, and includes vocabulary for modelling social web links.
- Workflow – workflow-related ontologies. Standards in this space remain relatively immature. The Common Workflow Language (CWL) has emerged as the most widely adopted tool-agnostic specification for describing computational workflows, and is a natural candidate for linking here. Other approaches to modelling workflows, such as Petri-Nets and various business process modelling ontologies, remain potentially relevant, though PROV-O may be a more durable choice than domain-specific alternatives for capturing workflow provenance in particular (Herschel et al. 2017).
- GitHub/Zenodo – links to source-code repositories and their citable archived releases. GitHub is the natural place to link an **Application**’s container (in our terminology **Box**) to its version-controlled source code, and to the history of commits that produced it. However, since a GitHub repository is mutable and its URL is not guaranteed to remain stable, Zenodo’s GitHub integration – which mints a DOI for an immutable snapshot of a repository whenever a release is tagged – provides the more durable, citable record better suited to linking from the Documentation table, and is increasingly the standard route by which research software is formally cited in publications.

In terms of maintenance, provision for making the links to external databases would require users to supply the relevant information, except where specifically supported databases provided an API allowing us to query them for possibly relevant data to link to.

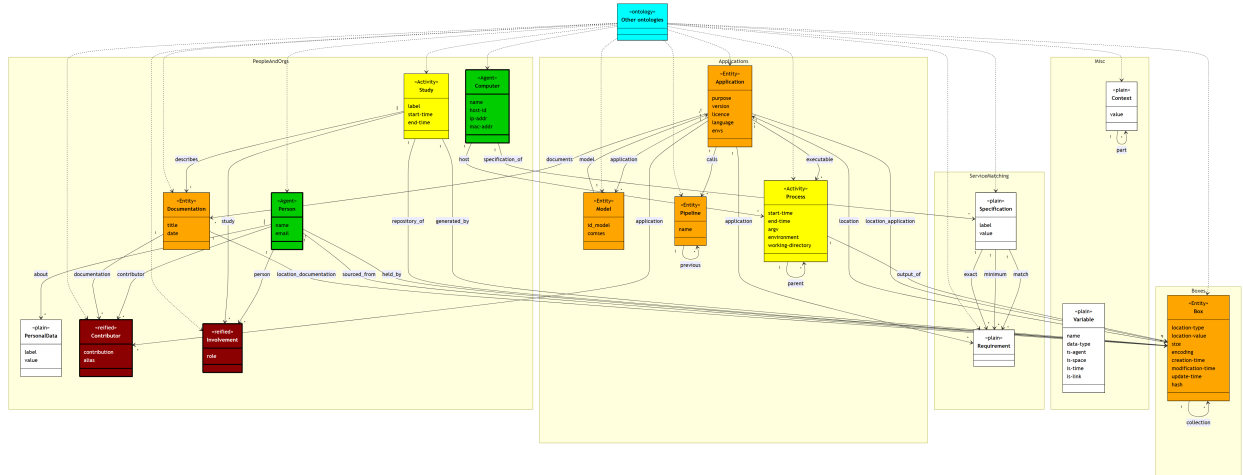


Figure 4: Subgraph showing possible tables that external ontologies could link to, and some relationships between them.

Fine grain

The fine grain metadata is information about the contents of files, including visualisations, and values of variables (fig. 5). Much of this has been covered already in the Analysis sec. section above, but here we show how Boxes are disaggregated from a provenance perspective through saying that **Values** and **Visualisations** are contained-in them, and from a workflow perspective through saying that **BoxTypes** have **Variables** and **VisualisationMethods** as their content. The provenance side can be populated automatically, but the workflow metadata would need to be provided by the user when describing an **Application** they were making available to the system: specifically, the user will need to describe the inputs and outputs of each application, and for each associated **BoxType**, provide details of **Variables** and **VisualisationMethods** given as content. For some file formats, autodetection may assist with populating the **Variable** table – e.g. given an example of a CSV file used by or generated by an application, a header row could be used to suggest names for **Variables** they supply.

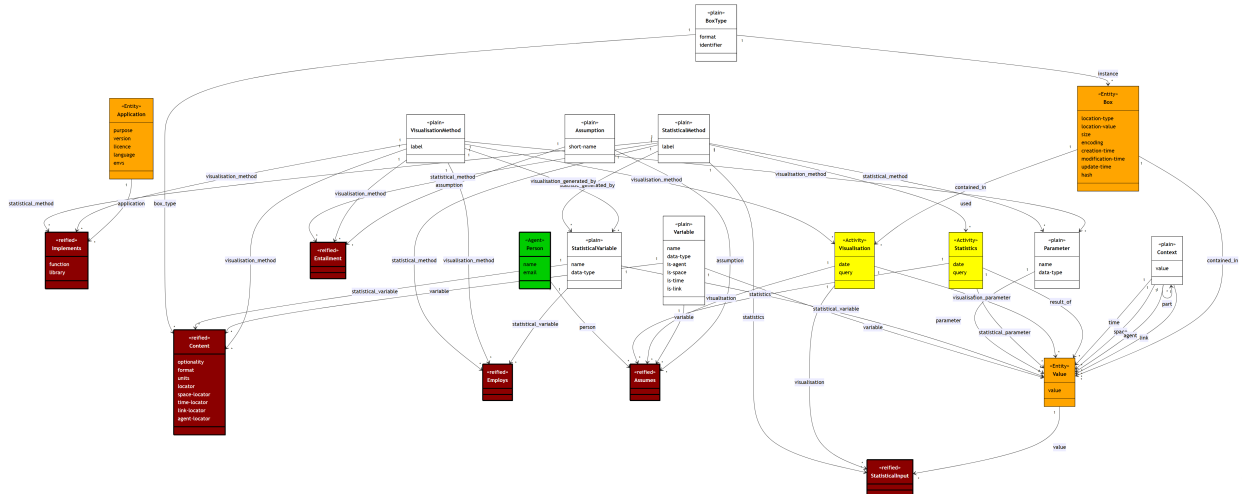


Figure 5: Subgraph showing the fine grain metadata tables

Folksonomy

Folksonomies are informal ontologies developed by a user communities, with tagging being a pretty much standard way in which this is achieved. It is not proposed to enrich that here. In fig. 6, the tables thought most likely to be of interest for users to tag are depicted, though this needn't be exhaustive. Each of the arcs from the Tag table would need a relational table to break the jmany-many relationship between tags and the concepts they are applied to.

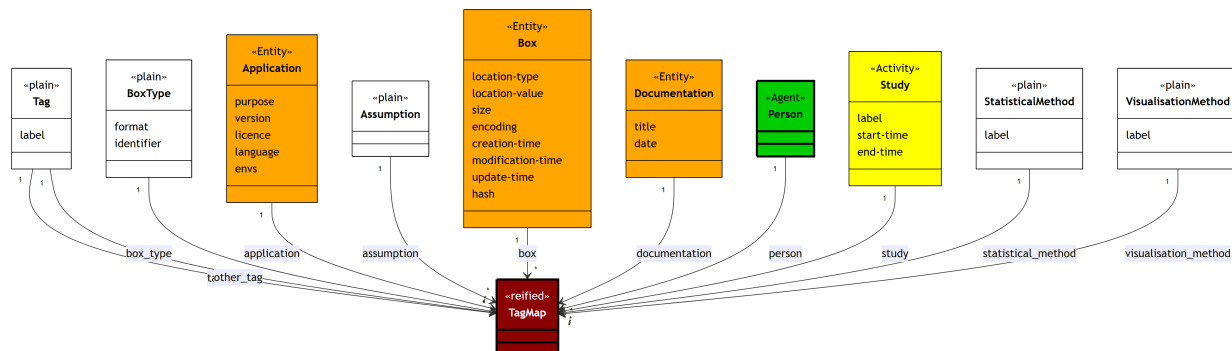


Figure 6: Folksonomy subgraph. Each of the arcs would itself be a reified table showing the application of Tags to tables.

Project

Project metadata (fig. 7) is largely for users to enter, and previously may have been regarded as unduly onerous (Edwards et al. 2014). However, it tells an important part of the story of a model from a Type 1 provenance (Pignotti et al. 2013) perspective, and with the advent of large language models, much of this metadata might be automatically generated. It is provided to facilitate users in understanding how simulation output data relates to publications (which would appear in the Documentation table), and specific pieces of work (Study table). The Study might be the most important table for users to complete, as this, by its association with Boxes allows collection of simulation outputs in useful groups.

Provenance

Provenance (fig. 8) is very much at the heart of the system, with an important role in recording how results from a model presented in journal articles are produced (Oliveira et al. 2018). Most of the tables here are autopopulated, as particularly for large scale analyses, user entry is an unrealistic goal. The system therefore needs to act as a wrapper around the scripts and tools (**Applications**) used to process the raw output from the simulation (which is the starting point of the MIRACLE project (Jin et al. 2017)) into the result used as part of an article. At the coarse grain, the central activity is the **Process**, which is a record of all relevant information of a process that ran on a **Computer**. This includes anything needed to replicate that **Process** under the same circumstances: the input files it used, command-line arguments, environment variables (essentially, anything that might affect its behaviour), and the output files it generated, the importance of which is illustrated by the challenges encountered when replicating the analysis of outputs from a social simulation of biodiversity incentivisation (Polhill et al. 2017).

The need for cross-platform support means that it is difficult to draw on standards for these. Different operating systems, and even different commands within the same operating systems, use different conventions for processing input on the command line, and input and output redirection. The **Argument** table attempts to record all relevant details about command-line arguments, allowing **ArgumentValues** to be inferred from a specific command-line instruction activating a **Process**. This is important in avoiding the need for users to supply too much information each time they run an **Application**.

Batch-file processing is assumed at the coarse grain; where processes involve user interaction that might affect

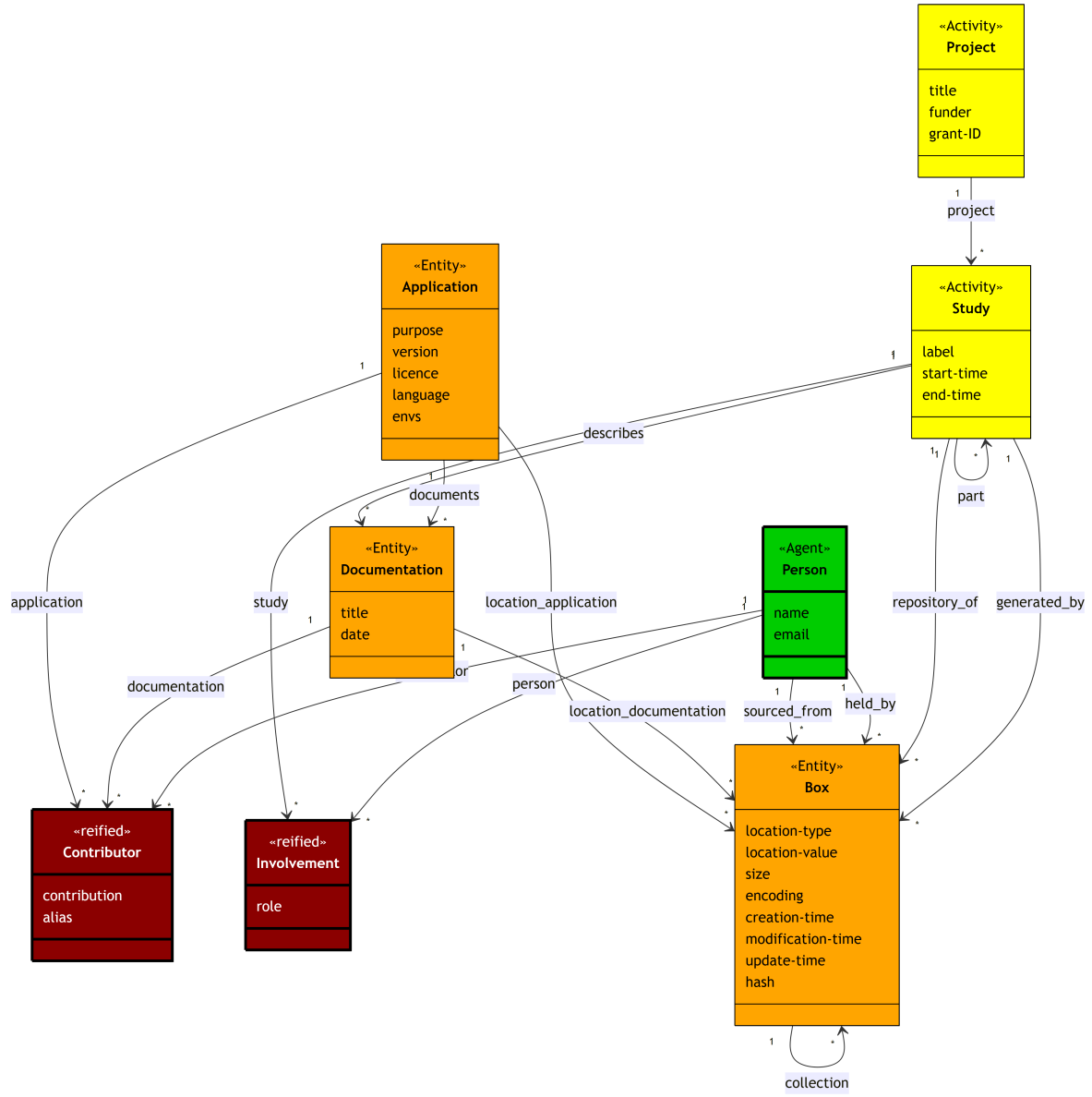


Figure 7: Subgraph capturing metadata about projects



the behaviour, automatic capture of provenance data would be extremely challenging. Whilst GUIs might be used during exploratory analysis of simulation output data, ultimately for reproducibility of results and keeping records of activities done (Pritchard and Wicenec 2025), these must ultimately be manifested as scripts that can be run in batch mode, once a decision is taken that a particular analysis step needs to be recorded as an application. **Applications** that only support GUI interaction can therefore not be supported (and arguably should be derided as virtually useless for any scientific endeavour – assuming repeatability is a desirable attribute of that work).

At the fine-grain, **Statistics** and **Visualisations** are the central activities. Again, although GUI interaction cannot be supported, command-line interaction potentially could. **Parameters** and **StatisticalInput** would need to be recorded, and the raw data on which the activities operate captured somehow in a query allowing the same set of data to be operated on. Recording the date at which the query was made is therefore important, especially if **Values** of **Variables** are subsequently populated that might affect the ability of future analyses to reuse the same data. A short-cut might be to store the date a **Value** was generated in the **Values** table, but since this table is expected to be ‘virtual’ (in that a supporting system would gather relevant data from the raw data files), the date can be captured from the dates of the **Processes** generating the **Boxes** in which the **Values** appear, or the date at which the **Statistics** were computed if the **Value** is a **StatisticalInput**.

Services

The services part of the graph depicted in fig. 9 provides a simple model intended to be used to determine whether an **Application** can be run on a **Computer**. This involves checking the **Requirement Specifications** of the **Application** (and recursively of any **Dependencies**) against the **Specifications** of a **Computer**. **Requirements** can be specified in one of three ways – for numeric **Requirement Specifications**, a ‘minimum’ **Requirement** must be exceeded by the **Specification** of the **Computer** (e.g. for RAM). For all types of **Requirement Specifications**, an ‘exact’ **Specification** must be equal (an example might be the OS – if the **Application** has very specific demands), whilst a ‘match’ **Specification** is a regular expression that the **Specification** of the **Computer** must match. The results are stored in the **Meets** reified relationship (or as a direct link between the two if this a graph database).

A more sophisticated implementation would wrap each **Application** in a web service. This would also be more secure if responsibility for providing **Applications** as web services was in the hands of their developers – users of the framework would then not be uploading **Applications** to their own system, but instead sending data for processing by other servers, and capturing metadata about these interactions. The services architecture could then draw much more heavily on web services ontologies, for example OWL-S (Martin et al. 2007) and WSDL (Christensen et al. 2001).

Workflow

The standard workflow model (fig. 10) provides prescriptive ‘recipes’ for undertaking specific procedures (Atkinson et al. 2017). Though we are interested in that here, and have provided a rudimentary sequence-based **Pipeline** table to handle that, we are also interested in providing support for exploring what ‘could be done’ given a current set of circumstances, and ‘what needs to be done’ before running an **Application**. For example, suppose we are interested in getting a **Value** for a particular **Variable**. Then we can see which **BoxTypes** have that **Variable** in their **Content**, and which **Applications** have the **BoxType** as a **Product**. If the **BoxTypes** those **Applications** Use have no instances, then we can find **Applications** that have the **Uses** **BoxTypes** as their **Product**, and so on. We can also search for **Pipelines** that have the required **Product** by exploring the **Products** of the last element of each **Pipeline**’s list-based structure, and checking the **Uses** of the first element.

This way, it should be possible, given a set of raw simulation outputs, to search for sequences of **Applications** to apply that generate a particular desired **Visualisation(Method)**. Using the provenance infrastructure, it would also be possible to explore how other users have chosen to do it in the past.

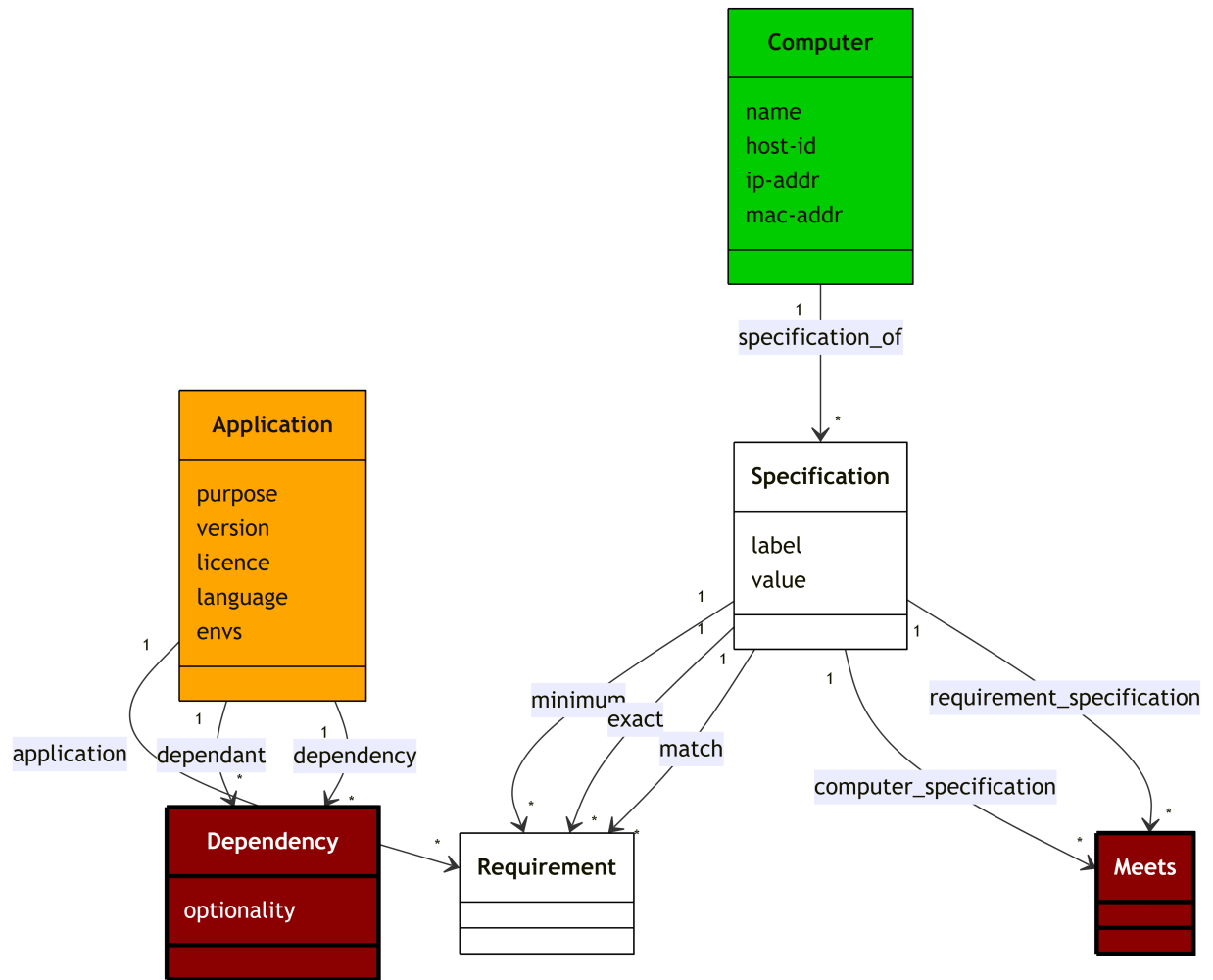


Figure 9: Services subgraph

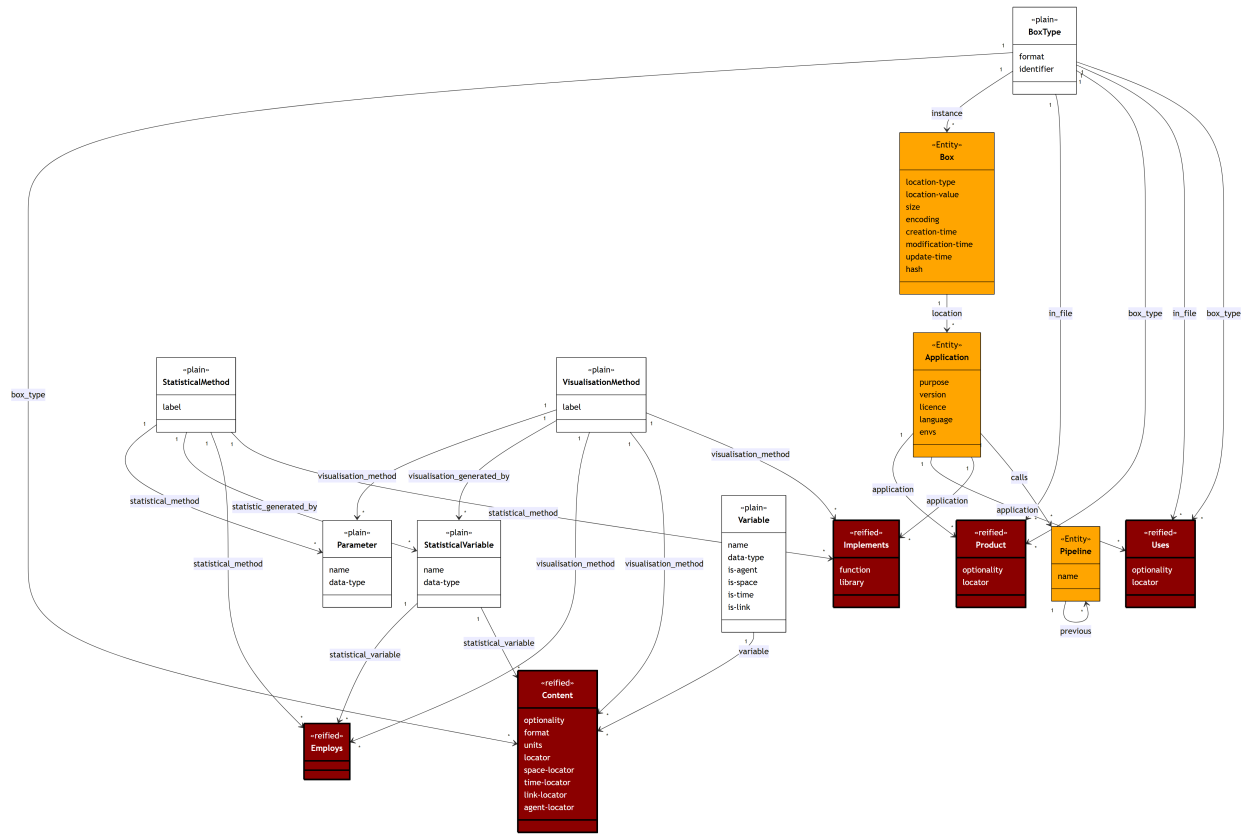


Figure 10: Workflow subgraph

Detailed specifications

All tables have uniquely specified ID fields as primary keys, unless they are associative tables (or direct links in the case of a graph database). All tables will have a name field. This is free form text and not always present. It identifies a relation, however it is not guaranteed to be unique. It is there to help primarily in readability when trying to extract information from this database and may contain a human readable label for the relation. For instance, for the definition of an argument, this would be the argument name. All tables will have a description field (dc:description). This field has been included to allow the introduction of free form documentary text. It is strongly advised that should the opportunity arise, then such text is supplied. The authors recommend this practice as it forms a useful means of documenting job control code in its own right, used to run these large scale models. Each table also has a creator/modifier field, and a corresponding created and modified date time field. Although most databases record this kind of metadata, we have made it explicit and expect these fields to be populated automatically. This is a deliberate attempt to make this interface database software agnostic. Note that date fields should be stored as strings conforming to the ISO 8601 date format, not in the native format of the database. This is recommended best practice according to the Dublin Core Metadata Initiative.^[1] The remaining attribute that is always included in each relation is the "about" column. This we have pinched directly from RDF and uniquely identifies a triple in RDF (Hartig and Zhao 2010). Although currently optional, we intend this to uniquely identify the relationship within our database. The form of this is normally some kind of IRI. This will allow inward referencing of the provenance and metadata resource from other standardised resource software that recognizes IRIs.

Attributes

Attribute	Description	Standards	Validation	Automation
name	A documentary name for the relation or entity	None	None. May or may not be present	None
description	Short text to use to summarise what the argument is	None	String	None
about	A unique resource identifier. This field will allow inward linking from any external resource that allow IRIs	RFC 3987 and RFC 4622	IRI	None
creator	Who created this particular set of attributes making up this relation	dc:creator	String	Should be done by the agency that has implemented the SSREPI interface. This will be the computer user
created	When this document was created	dc:created, ISO8601	logical date and time validation	Should be done by the agency that has implemented the SSREPI interface. This will be the computer user
modifier	Who created this particular set of attributes making up this relation	dc:creator	String	Should be done by the agency that has implemented the SSREPI interface. This will be the computer user
modified	When this document was created	dc:modified, ISO8601	Logical date and time validation. Must come after the date of creation.	Should be done by the agency that has implemented the SSREPI interface. This will be the computer user

Annotation

This is not shown in the above diagrams. This is database entry that can annotate any relation in this database. This can be used to flag certain properties, or record information about a given set of rows. Such properties will be recorded in a JSON format for the purposes validation.

Standards

JSON for the annotation values. SQL for row restriction.

Automation

There is an entry which documents a database query (if suitable) that will document the rows affected. It is up to the standard implementor whether such queries will be allowed to be performed automatically. We are recommending only statements that refer to the **Box** table should be allowed to stop the possibility of SQL injections.

Attributes

Attribute	Description	Standards	Validation	Automation
id_annotation	Unique key	None	Must be unique	Automated by the instantiating framework
json	json describing the values of the records being annotated.	None	Must be unique	Automated by the instantiating framework
sql	The SQL describing the range of records affected. This could be a single rows from single or multiple tables. BEWARE OF SQL CODE INJECTION when considering using this field	None	Must be unique	Automated by the instantiating framework

Relationships

None.

Application

Description

An **Application** is something that can be run by the user to generate or analyse simulation output.

Standards

PROV:Entity. Note the potential confusion. An **Application** is something that has the potential to be an activity (in the PROV sense) in the form of a **Process**. However, PROV only deals with the past, not with potential. The **Application** is a file somewhere, and hence an entity.

Automation

Most of this table is expected to be populated by the user.

Attributes

Field	Property	Value
ID_APPLICATION	Type	TEXT
	Description	Unique key
	Standards	None
	Validation	Must be unique
PURPOSE	Automation	Automated by the instatiating framework
	Type	TEXT
	Description	Description of the purpose of the application
	Standards	?
VERSION	Validation	Free text
	Automation	None
	Type	TEXT
	Description	Version ID for the application
LICENCE	Standards	dc:?
	Validation	String (e.g. CHAR(32))
	Automation	None
	Type	TEXT
LANGUAGE	Description	Software licence for the application
	Standards	dc:license
	Validation	Free text with the option to select from other entries
	Automation	None
ENVS	Type	TEXT
	Description	The environment variables that should be recorded when the application is run
	Standards	None
	Validation	List of strings
SEPARATOR	Automation	None
	Type	TEXT
	Description	Separator used when supplying arguments to the application
	Standards	None
	Validation	String
	Automation	None

Relationships

Field	Property	Value
REVISION	Type	TEXT
	Description	ID in Application table of Application this Application is a revision of, if any.
	Standards	PROV:wasDerivedFrom; dc:isVersionOF
	Validation	Must be an ID of an Application
MODEL	Null	If not a revision of anoth Application .
	Automation	None
	Type	TEXT
	Description	ID in Model table this Application is, if it is a Model .
LOCATION	Standards	None
	Validation	Must be an ID in Model .
	Null	If this Application is not a model.
	Automation	None
	Type	TEXT
	Description	ID_BOX in the Box table to use to find this Application .
	Standards	?
	Validation	Must be ID_BOX of a Box .
	Null	If the search for a Box for this Application has not been done.
	Automation	This should be populated automatically the first time the Application is requested on a host by finding the most local Box that references it, and storing that here.

If the most local **Box** is not on the current host, then the **Application** should be downloaded to the current host, and a new **Box** created for the location.

If the **Box ID** is no longer present, then the search should be repeated in the **Box** table. |

Argument

Description

A command-line argument accepted by an **Application**. Commands vary hugely in how they parse arguments on the command line, and this table needs to make clear how to build a command line that the **Application** can use. To be clear, a command line is a string of text that is given to a shell (DOS, bash, etc.) to initiate a batch job.

Standards

POSIX.1-2008 and equivalently IEEE Std 1003.1-2008/Cor 1-2013 are normative for Unix environments, but cannot be assumed even for Unix environments. Cross-platform support is in any case needed.

Automation

When building up a command-line for an **Application** with arguments during invocation of a **Process**, the system should do the following: - Let M be a map from integer to list of string. - For each flag, check with the user to see whether it should be set or cleared; if set, add the flag name to M, using the order as the key, or -1 if the order is null. Create an entry in **ArgumentValue** using ‘true’ or ‘false’ as the value according to whether or not the flag is set. - For each option, check with the user to see whether it should be used, and if so, provide an appropriate argument or arguments in accordance with the arity. Build a string for the option including its name and arguments, bearing in mind the separator and argsep values. Add the resulting string to M. - For each required argument, request a value or values from the user in accordance with the arity. Build a string accordingly and add to M. - Let K be a sorted list of keys of M. - Let C be a string. - For each key, join its list of strings with space and concatenate to C. - C contains the command-line argument string.

Attributes

Field	Property	Value
ID_ARGUMENT	Type	TEXT
	Description	Unique key
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework
TYPE	Type	TEXT
	Description	The kind of command line argument this is. Required arguments must be provided, typically don’t have a name, and are identified by their number. Options typically have a name, which if given stipulates some sort of value must be provided. Flags are options with arity 0.
	Standards	None
	Validation	One of “required”, “option”, or “flag”
	Automation	None
ORDER_VALUE	Type	INTEGER
	Description	A number used to indicate any order in which this Argument should appear in relation to other Arguments the Application accepts.
	Standards	None
	Validation	Non-negative integer or null if the order is unimportant.
	Automation	None
ASSIGNMENT_OPERATOR	Type	TEXT
	Description	Separator to use between argument name and value.
	Standards	None
	Validation	String

Field	Property	Value
NAME	Automation	None
	Type	TEXT
	Description	The name of the argument, including any grammar to indicate on the command-line that it is an argument (such as <code>—</code> or <code>-</code> or <code>/</code>).
SEPARATOR	Standards	None
	Validation	String
	Automation	None
SHORT_NAME	Type	TEXT
	Description	This is the character that indicates the argument name (e.g. <code>-input</code> , <code>-input, /input</code>).
	Standards	None
SHORT_SEPARATOR	Validation	String
	Automation	None
	Type	TEXT
DESCRIPTION	Description	This is the character that indicates the argument name (e.g. <code>-input</code> , <code>-input, /input</code>). for example in *nix system <code>-h</code> and <code>-help</code> are the same parameter.
	Standards	None
	Validation	String
ARITY	Automation	None
	Type	TEXT
	Description	Short text to use to summarise what the argument is
ARGSEP	Standards	None
	Validation	String
	Automation	None
RANGE	Type	TEXT
	Description	Number of arguments expected. Can be integer, <code>?</code> , <code>+</code> or <code>*</code> .
	Standards	None
	Validation	<code>?</code> , <code>+</code> or <code>*</code> , or integer string. Constraints apply.
	Automation	None
	Type	TEXT
	Description	Separator to use between arguments if arity > 1
	Standards	None
	Validation	String
	Automation	None
	Type	TEXT
	Description	Description of the range of any value to be supplied by the user
	Standards	None
	Validation	String
	Automation	None

Relationships

Field	Property	Value
APPLICATION	Type	TEXT
	Description	The application this argument applies to.
	Standards	None
	Validation	ID in the Application table.
	Null	Null if not an argument for an Application
VARIABLE	Automation	None
	Type	TEXT
	Description	The variable this argument relates to.
	Standards	None
	Validation	ID in the Variables table.
	Null	Null if not about a variable

Field	Property	Value
BOX_TYPE	Automation	None
	Type	TEXT
	Description	The BoxType this argument might be, if it is a file or somesuch.
	Standards	None
	Validation	ID in the BoxTypes table.
	Null	Null if this does not have a BoxType associated iwth it.
	Automation	None

Argument Value

Description

A value supplied for a command-line argument in a run of an **Application**.

Standards

None

Automation

Populated automatically when a **Process** is invoked.

Attributes

Field	Property	Value
HAS_VALUE	Type	TEXT
	Description	Value supplied, or true/false for flags
	Standards	None
	Validation	None
	Automation	Populated when the Process is invoked
FOR_PROCESS	Type	TEXT
	Description	ID in Process table of the Process this ArgumentValue applies to
	Standards	None
	Validation	Must be an ID of a Process
	Null	Not null
	Automation	Populated automatically when the Process is created
FOR_ARGUMENT	Type	TEXT
	Description	ID in Argument table of the Argument this value is for
	Standards	None
	Validation	Must be an ID of an Argument
	Null	Not null
BOX	Automation	Populated automatically when the Process is created
	Type	TEXT
	Description	ID in Box table of the box this value can be for
	Standards	None
	Validation	Must be an ID of a Box
	Null	Not null
	Automation	Populated automatically when the Process is created

Assumes

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Standards

None

Automation

This relationship may be inferred automatically from the use of **StatisticalMethod** or **VisualisationMethod**.

Relationships

Field	Property	Value
PERSON	Type	TEXT
	Description	ID in Person table of the Person making the assumption
	Standards	None
	Validation	Must be an ID of a Person
	Null	Not null
STATISTICS	Automation	None
	Type	TEXT
	Description	ID in Statistics table if assumption applies to a statistical computation
	Standards	None
	Validation	Must be an ID of a Statistics
VISUALISATION	Null	Null if Visualisation is not null
	Automation	None
	Type	TEXT
	Description	ID in Visualisation table if assumption applies to a Visualisation
	Standards	None
VARIABLE	Validation	Must be an ID of a Visualisation
	Null	Null if Statistics is not null
	Automation	None
	Type	TEXT
	Description	ID in Variable table of the variable to which the assumption applies
ASSUMPTION	Standards	None
	Validation	Must be an ID of a Variable
	Null	Not null
	Automation	None
	Type	TEXT
	Description	ID in Assumption table of the Assumption being made
	Standards	None
	Validation	Must be an ID of an Assumption
	Null	Not null
	Automation	None

Assumption

Description

An **Assumption** is a condition applied to a **Variable** for its proper application to a **Statistic**. (That **Statistic** being realised as an aggregation of the **Variable** to which the **Assumption** applies.) A **Person** makes an **Assumption** about a **Variable** (in the **Assumes** table), explicitly or implicitly, every time they compute the **Statistic** on it.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_ASSUMPTION	Type	TEXT
	Description	Short name for the assumption
	Standards	None
	Validation	Must be unique
	Automation	None

Box

Description

A **Box** is any data container (file, database, URI, etc.) used to store or reference data within the system.

Standards

PROV:Entity

Automation

A **Box** should be created automatically whenever data is accessed or generated by a **Process**. Metadata such as size, encoding, timestamps, and hash may be populated automatically using system tools (e.g. OS calls, HTTP headers, checksum utilities).

Attributes

Field	Property	Value
ID_BOX	Type	TEXT
	Description	Unique identifier for the Box
	Standards	None
	Validation	Primary key
	Automation	Generated automatically
LOCATION_TYPE	Type	TEXT
	Description	Type of resource the Box refers to (e.g. file, URI, database)
	Standards	?
	Validation	String describing access type
	Automation	Requires appropriate software depending on type
LOCATION_VALUE	Type	TEXT
	Description	The means of accessing the resource (path, URI, connection string, etc.)
	Standards	URI/IRI
	Validation	Depends on LOCATION-TYPE
	Automation	Parsed and used by system handlers
SIZE	Type	INTEGER
	Description	Size of the resource in bytes
	Standards	ISO/IEC 80000-13
	Validation	Non-negative integer or null
	Automation	Retrieved via OS or HTTP headers
ENCODING	Type	TEXT
	Description	Encoding of the resource
	Standards	MIME
	Validation	Valid MIME type or null
	Automation	Detected using system tools (e.g. file -I, HTTP headers)
CREATION_TIME	Type	TEXT
	Description	Time the resource was created
	Standards	ISO8601
	Validation	Datetime string
	Automation	Retrieved via OS stat or equivalent
MODIFICATION_TIME	Type	TEXT
	Description	Last modification time of the resource

Field	Property	Value
UPDATE_TIME	Standards	ISO8601
	Validation	Datetime string
	Automation	Retrieved via OS stat or HTTP headers
	Type	TEXT
	Description	Time the metadata for this Box was last updated
HASH	Standards	ISO8601
	Validation	Datetime string
	Automation	System maintained
	Type	TEXT
	Description	Hash of the resource content
INSTANCE	Standards	None
	Validation	Format: algorithm:encoding:value
	Automation	Generated using hashing tools (e.g. md5sum, sha256sum)
	Type	TEXT
	Description	ID in BoxType table describing the type of Box
	Standards	None
	Validation	Must be an ID of a BoxType
	Null	Not null
	Automation	None

Relationships

Field	Property	Value
LOCATION_APPLICATION	Type	TEXT
	Description	ID in Application table if this Box refers to an Application
	Standards	None
	Validation	Must be an ID of an Application .
	Null	Null if not applicable
LOCATION_DOCUMENTATION	Automation	None
	Type	TEXT
	Description	ID in Documentation table if this Box refers to Documentation
	Standards	None
	Validation	Must be an ID of Documentation
GENERATED_BY	Null	Null if not applicable
	Automation	None
	Type	TEXT
	Description	ID in Study table of the Study that generated this Box
	Standards	PROV:wasGeneratedBy
REPOSITORY_OF	Validation	Must be an ID of a Study
	Null	Null if not generated by a Study
	Automation	Set when Process produces output
	Type	TEXT
	Description	ID in Study table of Study this Box is a repository for
HELD_BY	Standards	None
	Validation	Must be an ID of a Study
	Null	Optional
	Automation	None
	Type	TEXT
SOURCED_FROM	Description	ID in Person table of person holding the Box
	Standards	PROV:wasAttributedTo
	Validation	Must be an ID of a Person
	Null	Optional
	Automation	None
OUTPUT_OF	Type	TEXT
	Description	ID in Process table of the Process that produced this Box

Field	Property	Value
COLLECTION	Standards	PROV:wasGeneratedBy
	Validation	Must be an ID of a Process
	Null	Null if not produced by a Process
	Automation	Automatically set during Process execution
	Type	TEXT
	Description	ID in Box table if this Box is part of another Box
	Standards	PROV:hadMember
	Validation	Must be an ID of a Box
	Null	Null if not part of a collection
	Automation	None

BoxType

Description

A **BoxType** defines the format and identification rules for Boxes, describing how their contents should be interpreted.

Standards

None

Automation

BoxTypes are expected to be defined by the user; identification rules may be applied automatically when inspecting **Box** contents.

Attributes

Field	Property	Value
ID_BOX_TYPE	Type	TEXT
	Description	Unique identifier for the BoxType
	Standards	None
	Validation	Must be unique
FORMAT	Automation	Automated by the instantiating framework
	Type	TEXT
	Description	Format of the Box contents
	Standards	MIME
IDENTIFIER	Validation	Valid MIME type
	Automation	None
	Type	TEXT
	Description	Rule used to identify whether a Box conforms to this BoxType (e.g. magic bytes, filename pattern)
	Standards	None
	Validation	Structured rule string
	Automation	None

Computer

Description

A **Computer** represents a machine on which a **Process** is executed.

Standards

PROV:Agent

Automation

Most values are expected to be automatically obtained from the operating system and network interfaces.

Attributes

Field	Property	Value
ID_COMPUTER	Type	TEXT
	Description	Unique key
	Standards	None
	Validation	Must be unique
HOST_ID	Automation	Automated by the instantiating framework
	Type	TEXT
	Description	Identifier for the host machine
	Standards	?
IP_ADDRESS	Validation	String
	Automation	Retrieved from the operating system
	Type	TEXT
	Description	IP address of the machine
MAC_ADDRESS	Standards	?
	Validation	Valid IP address string
	Automation	Retrieved from the network interface
	Type	TEXT
	Description	MAC address of the machine
	Standards	?
	Validation	Valid MAC address string
	Automation	Retrieved from the network interface

Content

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Content** table describes how a **Variable** (or **StatisticalVariable**) is located within a **Box**.

Standards

None

Automation

None

Attributes

Field	Property	Value
OPTIONALITY	Type	TEXT
	Description	Whether the Variable always appears or only appears depending on some condition
	Standards	None
	Validation	One of 'always' or 'depends'
LOCATOR	Automation	None
	Type	TEXT
	Description	How to locate the Variable value within the Box (e.g. row:X, column:Y, field:Z)
	Validation	Formatted rule string
	Automation	None

Field	Property	Value
TIME_LOCATOR	Type	TEXT
	Description	How to locate the temporal context (e.g. timestamps)
	Standards	None
	Validation	Formatted rule string
	Automation	None
LINK_LOCATOR	Type	TEXT
	Description	How to locate link identifiers (for relational data)
	Standards	None
	Validation	Formatted rule string
	Automation	None
AGENT_LOCATOR	Type	TEXT
	Description	How to locate agent identifiers
	Standards	None
	Validation	Formatted rule string
	Automation	None
STATISTICAL_VARIABLE	Type	TEXT
	Description	ID in StatisticalVariable table if this Content refers to a StatisticalVariable
	Standards	None
	Validation	Must be an ID of a StatisticalVariable
	Null	Null if VARIABLE is not null
	Automation	None

Relationships

Field	Property	Value
BOX_TYPE	Type	TEXT
	Description	ID in BoxType table of the BoxType this Content applies to
	Standards	None
	Validation	Must be an ID of a BoxType
	Null	Not null
	Automation	None
VARIABLE	Type	TEXT
	Description	ID in Variable table if this Content refers to a Variable
	Standards	None
	Validation	Must be an ID of a Variable
	Null	Null if STATISTICAL_VARIABLE is not null
	Automation	None
VISUALISATION_METHOD	Type	TEXT
	Description	ID in VisualisationMethod table if this Content is used for visualisation
	Standards	None
	Validation	Must be an ID of a VisualisationMethod
	Null	Optional
	Automation	None

Context

Description

The **Context** table provides contextual values such as time, space, agent, or link, which can be associated with **Values**.

Standards

None

Automation

Context entries may be created automatically when extracting or interpreting data from **Boxes**.

Attributes

Field	Property	Value
ID_CONTEXT	Type	TEXT
	Description	Unique identifier for the Context
	Standards	None
	Validation	Must be unique
VALUE	Automation	Automated by the instantiating framework
	Type	TEXT
	Description	Value of the context (e.g. time, space, agent, or link identifier)
	Standards	None
	Validation	String
	Automation	Derived from Content locators or data extraction

Relationships

Field	Property	Value
PART	Type	TEXT
	Description	ID in Context table indicating that this Context is part of another Context
	Standards	None
	Validation	Must be an ID of a Context
	Null	Null if this Context is not part of another
	Automation	None

Contributor

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Contributor** table records people who have contributed to **Applications** or **Documentation**, including how they are credited.

Standards

dc:contributor

Automation

None

Attributes

Field	Property	Value
CONTRIBUTION	Type	TEXT
	Description	Nature of the contribution made by the person
	Standards	dc:contributor
	Validation	String describing contribution type
	Automation	None

Field	Property	Value
ALIAS	Type	TEXT
	Description	Name or alias used for the contributor in the context of the Application or Documentation
	Standards	None
	Validation	String
	Automation	None

Relationships

Field	Property	Value
CONTRIBUTOR	Type	TEXT
	Description	ID in Person table of the contributor
	Standards	dc:contributor
	Validation	Must be an ID of a Person
	Null	Not null
	Automation	None
DOCUMENTATION	Type	TEXT
	Description	ID in Documentation table if the contribution relates to Documentation
	Standards	dc:relation
	Validation	Must be an ID of Documentation
	Null	Null if APPLICATION is not null
	Automation	None
APPLICATION	Type	TEXT
	Description	ID in Application table if the contribution relates to an Application
	Standards	dc:relation
	Validation	Must be an ID of an Application
	Null	Null if DOCUMENTATION is not null
	Automation	None

Dependency

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Dependency** table records that one **Application** depends on another, optionally under certain conditions.

Standards

None

Automation

A **Dependency** may be inferred automatically in some cases, but are generally provided by the user.

Attributes

Field	Property	Value
OPTIONALITY	Type	TEXT
	Description	Whether the dependency is required, optional, or conditional
	Standards	None
	Validation	One of 'required', 'optional', or a condition string
	Automation	None

Relationships

Field	Property	Value
DEPENDANT	Type	TEXT
	Description	ID in Application table of the Application that depends on another
	Standards	None
	Validation	Must be an ID of an Application
	Null	Not null
DEPENDENCY	Automation	None
	Type	TEXT
	Description	ID in Application table of the Application being depended upon
	Standards	None
	Validation	Must be an ID of an Application
	Null	Not null
	Automation	None

Documentation

Description

The **Documentation** table records documents that describe **Applications** or **Studies**.

Standards

dc:title; dc:created

Automation

None

Attributes

Field	Property	Value
ID_DOCUMENTATION	Type	TEXT
	Description	Unique identifier for the Documentation
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework
TITLE	Type	TEXT
	Description	Title of the documentation
	Standards	dc:title
	Validation	String
	Automation	None
DATE	Type	TEXT
	Description	Date the documentation was created
	Standards	dc:created; ISO8601
	Validation	Datetime string
	Automation	None

Relationships

Field	Property	Value
DOCUMENTS	Type	TEXT
	Description	ID in Application table of the Application this Documentation describes
	Standards	dc:relation

Field	Property	Value
DESCRIBES	Validation	Must be an ID of an Application
	Null	Null if REFERENCES is not null
	Automation	None
	Type	TEXT
	Description	ID in Study table of the Study this Documentation references
	Standards	dc:relation
	Validation	Must be an ID of a Study
	Null	Null if DOCUMENTS is not null
	Automation	None

Employs

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Employs** table records that a **StatisticalMethod** or **VisualisationMethod** employs a **StatisticalVariable**.

Standards

None

Automation

This relationship may be inferred automatically from the definitions of methods and variables.

Relationships

Field	Property	Value
STATISTICAL_METHOD	Type	TEXT
	Description	ID in StatisticalMethod table if the relationship involves a statistical method
	Standards	None
	Validation	Must be an ID of a StatisticalMethod
	Null	Null if VISUALISATION_METHOD is not null
	Automation	None
VISUALISATION_METHOD	Type	TEXT
	Description	ID in VisualisationMethod table if the relationship involves a visualisation method
	Standards	None
	Validation	Must be an ID of a VisualisationMethod
	Null	Null if STATISTICAL_METHOD is not null
	Automation	None
STATISTICAL_VARIABLE	Type	TEXT
	Description	ID in StatisticalVariable table of the variable being employed
	Standards	None
	Validation	Must be an ID of a StatisticalVariable
	Null	Not null
	Automation	None

Entailment

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Entailment** table records that a **StatisticalMethod** or **VisualisationMethod** entails an **Assumption**.

Standards

None

Automation

This relationship may be inferred automatically from the definitions of methods and assumptions.

Relationships

Field	Property	Value
STATISTICAL_METHOD	Type	TEXT
	Description	ID in StatisticalMethod table if the relationship involves a statistical method
	Standards	None
	Validation	Must be an ID of a StatisticalMethod
	Null	Null if VISUALISATION_METHOD is not null
	Automation	None
VISUALISATION_METHOD	Type	TEXT
	Description	ID in VisualisationMethod table if the relationship involves a visualisation method
	Standards	None
	Validation	Must be an ID of a VisualisationMethod
	Null	Null if STATISTICAL_METHOD is not null
	Automation	None
ASSUMPTION	Type	TEXT
	Description	ID in Assumption table of the Assumption that is entailed
	Standards	None
	Validation	Must be an ID of an Assumption
	Null	Not null
	Automation	None

Implements

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Implements** table records that an **Application** implements a **StatisticalMethod** or **VisualisationMethod**.

Standards

None

Automation

None

Attributes

Field	Property	Value
FUNCTION	Type	TEXT
	Description	Name of the function within the Application that implements the method
	Standards	None
	Validation	String
LIBRARY	Automation	None
	Type	TEXT
	Description	Name of the library within which the function is found
	Standards	None
	Validation	String
	Automation	None

Relationships

Field	Property	Value
STATISTICAL_METHOD	Type	TEXT
	Description	ID in StatisticalMethod table if the implementation is for a statistical method
	Standards	None
	Validation	Must be an ID of a StatisticalMethod
	Null	Null if VISUALISATION_METHOD is not null
VISUALISATION_METHOD	Automation	None
	Type	TEXT
	Description	ID in VisualisationMethod table if the implementation is for a visualisation method
	Standards	None
	Validation	Must be an ID of a VisualisationMethod
APPLICATION	Null	Null if STATISTICAL_METHOD is not null
	Automation	None
	Type	TEXT
	Description	ID in Application table of the Application implementing the method
	Standards	None
	Validation	Must be an ID of an Application
	Null	Not null
	Automation	None

Input

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Input** table records the **Boxes** that are used as input to a **Process**, including how they are used.

Standards

PROV:used

Automation

Populated automatically when a **Process** is invoked and its inputs are identified.

Attributes

Field	Property	Value
USAGE	Type	TEXT
	Description	Indicates how the input is used by the Process
	Standards	None
	Validation	One of 'dependency' or 'data'
	Automation	Set automatically based on the role of the input

Relationships

Field	Property	Value
PROCESS	Type	TEXT
	Description	ID in Process table of the Process using the input
	Standards	PROV:used
	Validation	Must be an ID of a Process
	Null	Not null
BOX	Automation	Populated automatically when the Process is created
	Type	TEXT
	Description	ID in Box table of the Box being used as input
	Standards	PROV:Entity
	Validation	Must be an ID of a Box
	Null	Not null
	Automation	Populated automatically when inputs are resolved

Involvement

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Involvement** table records the involvement of a **Person** in a **Study**, including their role.

Standards

PROV:wasAssociatedWith

Automation

None

Attributes

Field	Property	Value
ROLE	Type	TEXT
	Description	Role of the Person in the Study
	Standards	None
	Validation	String
	Automation	None

Relationships

Field	Property	Value
PERSON	Type	TEXT
	Description	ID in Person table of the Person involved
	Standards	PROV:Agent
	Validation	Must be an ID of a Person
	Null	Not null
STUDY	Automation	None
	Type	TEXT
	Description	ID in Study table of the Study the Person is involved in
	Standards	PROV:Activity
	Validation	Must be an ID of a Study
	Null	Not null
	Automation	None

Meets

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Meets** table records that a **Computer** meets a specified **Specification**.

Standards

None

Automation

None

Relationships

Field	Property	Value
COMPUTER_SPECIFICATION	Type	TEXT
	Description	ID in Specification table describing the specification met by the Computer
	Standards	None
	Validation	Must be an ID of a Specification
	Null	Not null
REQUIREMENT_SPECIFICATION	Automation	None
	Type	TEXT
	Description	ID in Specification table defining the required specification being met
	Standards	None
	Validation	Must be an ID of a Specification
	Null	Not null
	Automation	None

Model

Description

The **Model** table identifies **Applications** that represent models, optionally linking to external model registries.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_MODEL	Type	TEXT
	Description	Unique identifier for the Model
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework
COMSES	Type	TEXT
	Description	Reference to an entry in the CoMSES-Net model repository
	Standards	None
	Validation	String or identifier
	Null	Optional
	Automation	None

Relationships

Field	Property	Value
APPLICATION	Type	TEXT
	Description	ID in Application table of the Application that is a Model
	Standards	None
	Validation	Must be an ID of an Application
	Null	Not null
	Automation	None

Parameter

Description

The **Parameter** table records parameters used by **StatisticalMethod** or **VisualisationMethod**.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_PARAMETER	Type	TEXT
	Description	Unique identifier for the Parameter
	Standards	None
	Validation	Must be unique
DATA_TYPE	Automation	Automated by the instantiating framework
	Type	TEXT

Field	Property	Value
	Description	Data type of the parameter
	Standards	XSD
	Validation	Valid XSD data type or URI
	Automation	None

Relationships

Field	Property	Value
STATISTICAL_METHOD	Type	TEXT
	Description	ID in StatisticalMethod table if the parameter applies to a statistical method
	Standards	None
	Validation	Must be an ID of a StatisticalMethod
	Null	Null if VISUALISATION_METHOD is not null
	Automation	None
VISUALISATION_METHOD	Type	TEXT
	Description	ID in VisualisationMethod table if the parameter applies to a visualisation method
	Standards	None
	Validation	Must be an ID of a VisualisationMethod
	Null	Null if STATISTICAL_METHOD is not null
	Automation	None

Person

Description

The **Person** table records individuals associated with the system, such as users, contributors, or data owners.

Standards

PROV:Agent; FOAF

Automation

None

Attributes

Field	Property	Value
ID_PERSON	Type	TEXT
	Description	Unique identifier for the Person
	Standards	FOAF
	Validation	Must be unique
	Automation	Automated by the instantiating framework
EMAIL	Type	TEXT
	Description	Email address of the Person
	Standards	FOAF
	Validation	Valid email address string
	Automation	None

PersonalData

Description

The **PersonalData** table records additional pieces of information about a **Person** as label–value pairs.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_PERSONAL_DATA	Type	TEXT
	Description	Unique identifier for the PersonalData entry
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework
CATEGORY	Type	TEXT
	Description	Category describing the type of personal data
	Standards	None
	Validation	String
	Automation	None
VALUE	Type	TEXT
	Description	Value of the personal data
	Standards	None
	Validation	String
	Automation	None

Relationships

Field	Property	Value
ABOUT	Type	TEXT
	Description	ID in Person table of the Person this data is about
	Standards	None
	Validation	Must be an ID of a Person
	Null	Not null
	Automation	None

Pipeline

Description

The **Pipeline** table records sequences of **Applications**, allowing workflows to be defined where one **Application** calls another.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_PIPELINE	Type	TEXT
	Description	Unique identifier for the Pipeline
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework

Relationships

Field	Property	Value
CALLS	Type	TEXT
	Description	ID in Application table of the Application called by this Pipeline step
	Standards	None
	Validation	Must be an ID of an Application
	Null	Not null
PREVIOUS	Automation	None
	Type	TEXT
	Nullable	True
	Description	ID in Pipeline table of the previous Pipeline step
	Standards	None
	Validation	Must be an ID of a Pipeline
	Null	Null if this is the first step
	Automation	None

Process

Description

The **Process** table records an execution of an **Application**, including when and how it was run.

Standards

PROV:Activity

Automation

Entries are created automatically when an **Application** is executed, capturing runtime details.

Attributes

Field	Property	Value
ID_PROCESS	Type	TEXT
	Description	Unique identifier for the Process
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework
START_TIME	Type	TEXT
	Description	Time at which the Process started
	Standards	ISO8601
	Validation	Datetime string

Field	Property	Value
END_TIME	Automation	Captured automatically at process start
	Type	TEXT
	Description	Time at which the Process ended
	Standards	ISO8601
ARGV	Validation	Datetime string
	Automation	Captured automatically at process completion
	Type	TEXT
	Description	Command-line string used to invoke the Application
ENVIRONMENT	Standards	POSIX
	Validation	String
	Automation	Constructed automatically from Argument and ArgumentValue
	Type	TEXT
WORKING_DIR	Description	Environment variables used during execution
	Standards	POSIX
	Validation	List of strings
	Automation	Captured from runtime environment
	Type	TEXT
	Description	Working directory from which the Process was executed
	Standards	POSIX
	Validation	Valid file path
	Automation	Captured automatically from runtime

Relationships

Field	Property	Value
EXECUTABLE	Type	TEXT
	Description	ID in Application table of the Application that was executed
	Standards	PROV:used
	Validation	Must be an ID of an Application
SOME_USER	Null	Not null
	Automation	Set when the Process is created
	Type	TEXT
	Description	ID in User table of the user who initiated the Process
HOST	Standards	PROV:wasAssociatedWith
	Validation	Must be an ID of a User
	Null	Not null
	Automation	Captured from system user context
	Type	TEXT
	Description	ID in Computer table of the machine on which the Process ran
	Standards	PROV:wasAssociatedWith
	Validation	Must be an ID of a Computer
PARENT	Null	Not null
	Automation	Captured from system environment
	Type	TEXT
	Description	ID in Process table of the parent Process , if any
	Standards	PROV:wasInformedBy
	Validation	Must be an ID of a Process
	Null	Null if no parent process
	Automation	Set if Process is spawned by another

Product

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Product** table describes the types of output that an **Application** can produce, including how those outputs are located.

Standards

None

Automation

Some **Products** may be inferred automatically based on **Application** execution and generated **Boxes**.

Attributes

Field	Property	Value
OPTIONALITY	Type	TEXT
	Description	Whether the Product is always produced or only under certain conditions
	Standards	None
	Validation	One of 'always' or 'depends'
LOCATOR	Automation	None
	Type	TEXT
	Description	Description of how to locate the Product output
	Standards	None
	Validation	Formatted rule string
	Automation	None

Relationships

Field	Property	Value
APPLICATION	Type	TEXT
	Description	ID in Application table of the Application producing this Product
	Standards	None
	Validation	Must be an ID of an Application
	Null	Not null
	Automation	None
BOX_TYPE	Type	TEXT
	Description	ID in BoxType table describing the type of Box produced
	Standards	None
	Validation	Must be an ID of a BoxType
	Null	Not null
	Automation	None
IN_FILE	Type	TEXT
	Description	ID in BoxType table if the Product is contained within another file
	Standards	None
	Validation	Must be an ID of a BoxType
	Null	Null if LOCATOR is not 'in-file'
	Automation	None

Project

Description

The **Project** table records **Projects**, which are collections of **Studies** and may include funding and organisational information.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_PROJECT	Type	TEXT
	Description	Unique identifier for the Project
	Standards	None
	Validation	Must be unique
TITLE	Automation	Automated by the instantiating framework
	Type	TEXT
	Description	Title of the Project
	Standards	dc:title
FUNDER	Validation	String
	Automation	None
	Type	TEXT
	Description	Organisation or body funding the Project
GRANT_ID	Standards	dc:publisher
	Validation	String
	Automation	None
	Type	TEXT
	Description	Identifier of the grant funding the Project
	Standards	None
	Validation	String
	Automation	None

Relationships

Field	Property	Value
STUDY	Type	TEXT
	Description	A Study is a piece of work at some level of aggregation.
	Standards	None
	Validation	An ID in the Study table.
	Null	Not Null
	Automation	None

Requirement

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **Requirement** table records requirements that an **Application** has with respect to **Specifications**, including exact, minimum, or matching constraints.

Standards

None

Automation

None

Relationships

Field	Property	Value
APPLICATION	Type	TEXT

Field	Property	Value
MATCH	Description	ID in Application table of the Application that has the requirement
	Standards	None
	Validation	Must be an ID of an Application
	Null	Not null
	Automation	None
MINIMUM	Type	TEXT
	Description	ID in Specification table specifying values that must match
	Standards	None
	Validation	Must be an ID of a Specification
	Null	Null if EXACT or MINIMUM is used
EXACT	Automation	None
	Type	TEXT
	Description	ID in Specification table specifying minimum acceptable values
	Standards	None
	Validation	Must be an ID of a Specification
	Null	Null if EXACT or MATCH is used
	Automation	None
	Type	TEXT
	Description	ID in Specification table specifying exact required values
	Standards	None
	Validation	Must be an ID of a Specification
	Null	Null if MATCH or MINIMUM is used
	Automation	None

Specification

Description

The **Specification** table defines labelled specification values, typically used to describe properties of Computers or requirements of Applications.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_SPECIFICATION	Type	TEXT
	Description	Unique identifier for the Specification
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework
CATEGORY	Type	TEXT
	Description	Category describing the type of specification
	Standards	None
	Validation	String
	Automation	None
VALUE	Type	TEXT
	Description	Value of the specification
	Standards	None
	Validation	String
	Automation	None

Relationships

Field	Property	Value
SPECIFICATION_OF	Type	TEXT
	Description	ID in Computer table of the Computer this Specification describes
	Standards	None
	Validation	Must be an ID of a Computer
	Null	Optional
	Automation	None

StatisticalInput

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **StatisticalInput** table records the input **Variables** or **Boxes** used by a **StatisticalMethod** when performing a statistical computation.

Standards

PROV:used

Automation

Inputs may be inferred automatically when a **Statistics** computation is executed.

Relationships

Field	Property	Value
STATISTICS	Type	TEXT
	Description	ID in StatisticalMethod table of the method using the input
	Standards	PROV:used
	Validation	Must be an ID of a StatisticalMethod
	Null	Not null
	Automation	None
VISUALISATION	Type	TEXT
	Description	ID in Visualisations table of the Visualisation used as input to the method
	Standards	PROV:Entity
	Validation	Must be an ID of a Visualisation
	Null	Null if VARIABLE is not null
	Automation	Resolved automatically where possible
VALUE	Type	TEXT
	Description	ID in Value table of the Variable used as input to the method
	Standards	None
	Validation	Must be an ID of a Value
	Null	Null if Value is not null
	Automation	Resolved automatically from Content definitions

StatisticalMethod

Description

The **StatisticalMethod** table records statistical methods that may be applied to data to generate **StatisticalVariables**.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_STATISTICAL_METHOD	Type	TEXT
	Description	Unique identifier for the StatisticalMethod
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework

StatisticalVariable

Description

The **StatisticalVariable** table records variables that are generated by applying a **StatisticalMethod**.

Standards

None

Automation

StatisticalVariables may be created automatically as outputs of statistical computations.

Attributes

Field	Property	Value
ID_STATISTICAL_VARIABLE	Type	TEXT
	Description	Unique identifier for the StatisticalVariable
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework
DATA_TYPE	Type	TEXT
	Description	Data type of the StatisticalVariable
	Standards	XSD
	Validation	Valid XSD data type or URI
	Automation	None

Relationships

Field	Property	Value
STATISTIC_GENERATED_BY	Type	TEXT
	Description	ID in StatisticalMethod table of the method that generates this StatisticalVariable
	Standards	PROV:wasGeneratedBy
	Validation	Must be an ID of a StatisticalMethod

Field	Property	Value
VISUALISATION_GENERATED_BY	Null	Not null
	Automation	None
	Type	TEXT
	Description	ID in VisualisationMethod table of the method that generates this Visualisation
	Standards	PROV:wasGeneratedBy
	Validation	Must be an ID of a VisualisationMethod .
	Null	Not null
	Automation	None

Statistics

Description

The **Statistics** table records statistical computations, including when they were performed and how input data was selected.

Standards

PROV:Activity

Automation

Entries may be created automatically when statistical computations are performed.

Attributes

Field	Property	Value
ID_STATISTICS	Type	TEXT
	Description	Unique identifier for the Statistics computation
	Standards	None
	Validation	Must be unique
DATE	Automation	Automated by the instantiating framework
	Type	DATE
	Description	Date the statistical computation was performed
	Standards	ISO8601
QUERY	Validation	Datetime string
	Automation	Set automatically at execution time
	Type	TEXT
	Description	Query used to select the data for the statistical computation
	Standards	None
	Validation	Formatted string
	Automation	None

Relationships

Field	Property	Value
USED	Type	TEXT
	Description	ID in StatisticalMethod table of the method used for the computation
	Standards	PROV:used
	Validation	Must be an ID of a StatisticalMethod
	Null	Not null
	Automation	Set when the Statistics entry is created

Study

Description

The **Study** table records units of scientific work, representing collections of **Processes**, data, and outputs.

Standards

PROV:Activity

Automation

Some fields may be populated automatically when **Processes** are grouped into **Studies**.

Attributes

Field	Property	Value
ID_STUDY	Type	TEXT
	Description	Unique identifier for the Study
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework
TITLE	Type	TEXT
	Description	Title or name of the Study
	Standards	dc:title
	Validation	String
	Automation	None
START_TIME	Type	DATE
	Description	Start time of the Study
	Standards	ISO8601
	Validation	Datetime string
	Automation	May be inferred from earliest Process
END_TIME	Type	DATE
	Description	End time of the Study
	Standards	ISO8601
	Validation	Datetime string
	Automation	May be inferred from latest Process

Relationships

Field	Property	Value
PROJECT	Type	TEXT
	Description	ID in Project table of the Project this Study belongs to
	Standards	None
	Validation	Must be an ID of a Project
	Null	Optional
	Automation	None
PART	Type	TEXT
	Description	ID in Study table if this Study is part of another Study
	Standards	PROV:wasPartOf
	Validation	Must be an ID of a Study
	Null	Null if not part of another Study
	Automation	None

Tag

Description

The **Tag** table records tags that can be used to classify and annotate other entities in the system.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_TAG	Type	TEXT
	Description	Unique identifier for the Tag
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework

TagMap

This is reification. In the graph database, this will appear as an edge between two entites. In a relational database, this will be a relation which allows the enumeration of the many-to-many linkage.

Description

The **TagMap** table records the association of **Tags** with other entities such as **Applications**, **Boxes**, **Documentation**, **Studies**, and **Methods**.

Standards

None

Automation

Tags may be applied manually by users or inferred automatically based on metadata and usage.

Relationships

Field	Property	Value
TAG	Type	TEXT
	Description	ID in Tag table of the Tag being applied
	Standards	None
	Validation	Must be an ID of a Tag
	Null	Not null
APPLICATION	Automation	None
	Type	TEXT
	Description	ID in Application table if the Tag applies to an Application
	Standards	None
	Validation	Must be an ID of an Application
ASSUMPTION	Null	Null if not applicable
	Automation	None
	Type	TEXT
	Description	ID in Assumptoin table if the Tag applies to an Assumption .
	Standards	None
BOX	Validation	Must be an ID of an Assumption
	Null	Null if not applicable
	Automation	None
	Type	TEXT

Field	Property	Value
BOX_TYPE	Description	ID in Box table if the Tag applies to a Box
	Standards	None
	Validation	Must be an ID of a Box
	Null	Null if not applicable
	Automation	None
DOCUMENTATION	Type	TEXT
	Description	ID in BoxType table if the Tag applies to a BoxType
	Standards	None
	Validation	Must be an ID of a BoxType
	Null	Null if not applicable
OTHER_TAG	Automation	None
	Type	TEXT
	Description	ID in Documentation table if the Tag applies to Documentation
	Standards	None
	Validation	Must be an ID of a Documentation
PERSON	Null	Null if not applicable
	Automation	None
	Type	TEXT
	Description	ID in Tag table if the Tag relates to another Tag
	Standards	None
STATISTICAL_METHOD	Validation	Must be an ID of a Tag
	Null	Null if not applicable
	Automation	None
	Type	TEXT
	Description	ID in Person table is the Person in the Person table.
STUDY	Standards	None
	Validation	Must be an ID of a Tag
	Null	Null if not applicable
	Automation	None
	Type	TEXT
VISUALISATION_METHOD	Description	ID in Study table if the Tag applies to a Study
	Standards	None
	Validation	Must be an ID of a Study
	Null	Null if not applicable
	Automation	None
	Type	TEXT
	Description	ID in VisualisationMethod table if the Tag applies to a visualisation method
	Standards	None
	Validation	Must be an ID of a VisualisationMethod
	Null	Null if not applicable
	Automation	None

User

Description

The **User** table records system user accounts, linking operating system user information to a **Person**.

Standards

PROV:Agent

Automation

Most values are obtained automatically from the operating system.

Attributes

Field	Property	Value
ID_USER	Type	TEXT
	Description	Unique identifier for the User
	Standards	None
	Validation	Must be unique
HOME_DIR	Automation	Automated by the instantiating framework
	Type	TEXT
	Description	Home directory of the user
	Standards	POSIX
	Validation	Valid file path
	Automation	Retrieved from the operating system

Relationships

Field	Property	Value
ACCOUNT_OF	Type	TEXT
	Description	ID in Person table of the Person this User account belongs to
	Standards	PROV:actedOnBehalfOf
	Validation	Must be an ID of a Person
	Null	Not null
	Automation	Resolved from system/user configuration

Uses

Description

The **Uses** table records that an **Application** uses a **BoxType** as an input or dependency.

Standards

PROV:used

Automation

May be inferred automatically based on **Application** execution and detected **Inputs**.

Attributes

Field	Property	Value
LOCATOR	Type	TEXT
	Description	Description of how to locate the Uses input
	Standards	None
	Validation	Formatted rule string
	Automation	None

Relationships

Field	Property	Value
APPLICATION	Type	TEXT
	Description	ID in Application table of the Application that uses the input
	Standards	PROV:Activity
	Validation	Must be an ID of an Application
	Null	Not null
BOX_TYPE	Automation	None
	Type	TEXT
	Description	ID in BoxType table of the type of Box that is used
	Standards	PROV:Entity
	Validation	Must be an ID of a BoxType
IN_FILE	Null	Not null
	Automation	None
	Type	TEXT
	Description	ID in BoxType table if the Uses input is contained within another file
	Standards	None
	Validation	Must be an ID of a BoxType
	Null	Null if LOCATOR is not 'in-file'
	Automation	None

Value

Description

The **Value** table represents values of **Variables** or **StatisticalVariables**. It is a virtual table whose entries are retrieved from **Boxes** rather than stored directly.

Standards

PROV:Entity

Automation

Values are not stored explicitly; they are retrieved dynamically from **Boxes** based on **Content** specifications.

Attributes

Field	Property	Value
ID_VALUE	Type	TEXT
	Constraint	PRIMARY KEY
	Description	Identifier or representation of the value
	Standards	None
	Validation	This should be a unique string
UNITS	Automation	Retrieved from underlying data in Box
	Type	TEXT
	Description	The units of the value.
	Standards	None
	Validation	None
FORMAT	Automation	None.
	Type	TEXT
	Description	ID in Variable table of the Variable this Value corresponds to
	Standards	None
	Validation	Values are not always numbers.
	Automation	None.

Relationships

Field	Property	Value
VARIABLE	Type	TEXT
	Description	ID in Variable table of the Variable this Value corresponds to
	Standards	None
	Validation	Must be an ID of a Variable
	Null	Null if STATISTICAL_VARIABLE is used
STATISTICAL_VARIABLE	Automation	Resolved via Content definitions
	Type	TEXT
	Description	ID in StatisticalVariable table if this Value is the result of a statistical computation
	Standards	None
	Validation	Must be an ID of a StatisticalVariable
PARAMETER	Null	Null if VARIABLE is used
	Automation	Resolved via statistical processing
	Type	TEXT
	Description	ID in Parameter table if this Value corresponds to a parameter
	Standards	None
STATISTICAL_PARAMETER	Validation	Must be an ID of a Parameter
	Null	Optional
	Automation	Resolved during method execution
	Type	TEXT
	Description	ID in Statistics table if this Value corresponds to a Statistic
VISUALISATION_PARAMETER	Standards	None
	Validation	Must be an ID of a Statistic
	Null	Optional
	Automation	Resolved during method execution
	Type	TEXT
RESULT_OF	Description	ID in the Visualisation table if this Value corresponds to a Visualisation
	Standards	None
	Validation	Must be an ID of a Visualisation
	Null	Optional
	Automation	Resolved during method execution
TIME	Type	TEXT
	Description	ID in Context table representing the time associated with the Value
	Standards	None
	Validation	Must be an ID of a Context
	Null	Optional
SPACE	Automation	Derived from Content locators
	Type	TEXT
	Description	ID in Context table representing the spatial context of the Value
	Standards	None
	Validation	Must be an ID of a Context
AGENT	Null	Optional
	Automation	Derived from Content locators
	Type	TEXT
	Description	ID in Context table representing the agent associated with the Value
	Standards	None
LINK	Validation	Must be an ID of a Context
	Null	Optional
	Automation	Derived from Content locators
	Type	TEXT
	Description	ID in Context table representing link relationships associated with the Value
	Standards	None
	Validation	Must be an ID of a Context
	Null	Optional

Field	Property	Value
CONTAINED_IN	Automation	Derived from Content locators
	Type	TEXT
	Description	ID in Box table from which the Value is retrieved
	Standards	PROV:Entity
	Validation	Must be an ID of a Box
	Null	Not null
	Automation	Determined from data source

Variable

Description

The **Variable** table records variables that describe data, including their name, type, and roles such as time, space, agent, or link.

Standards

None

Automation

Variables are generally defined by the user; some roles may be inferred during data processing.

Attributes

Field	Property	Value
ID_VARIABLE	Type	TEXT
	Description	Identifier of the variable
	Standards	None
	Validation	Must be unique
	Automation	None
NAME	Type	TEXT
	Description	Name of the variable
	Standards	None
	Validation	String
	Automation	None
DATA_TYPE	Type	TEXT
	Description	Data type of the variable
	Standards	XSD
	Validation	Valid XSD data type or URI
	Automation	None
IS_AGENT	Type	INTEGER
	Description	Indicates whether this variable represents an agent identifier
	Standards	None
	Validation	True or False
	Automation	None
IS_LINK	Type	INTEGER
	Description	Indicates whether this variable represents a link identifier
	Standards	None
	Validation	True or False
	Automation	None
IS_SPACE	Type	INTEGER
	Description	Indicates whether this variable represents spatial information
	Standards	None
	Validation	True or False
	Automation	None
IS_TIME	Type	INTEGER
	Description	Indicates whether this variable represents temporal information
	Standards	None
	Validation	True or False

Field	Property	Value
	Automation	None

Visualisation

Description

The **Visualisation** table records visualisations generated from data, including when they were created and how the data was selected.

Standards

PROV:Activity

Automation

Entries may be created automatically when a visualisation is generated.

Attributes

Field	Property	Value
ID_VISUALISATION	Type	TEXT
	Description	Unique identifier for the Visualisation
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework
DATE	Type	DATE
	Description	Date the Visualisation was created
	Standards	ISO8601
	Validation	Datetime string
QUERY	Automation	Set automatically at creation time
	Type	TEXT
	Description	Query used to select data for the Visualisation
	Standards	None
	Validation	Formatted string
	Automation	None

Relationships

Field	Property	Value
VISUALISATION_METHOD	Type	TEXT
	Description	ID in VisualisationMethod table of the method used to generate the Visualisation
	Standards	PROV:used
	Validation	Must be an ID of a VisualisationMethod
	Null	Not null
	Automation	Set when the Visualisation is created
CONTAINED_IN	Type	TEXT
	Description	ID in Box table of the Box containing the Visualisation
	Standards	PROV:Entity
	Validation	Must be an ID of a Box
	Null	Optional
	Automation	Set if output is stored in a Box

VisualisationMethod

Description

The **VisualisationMethod** table records methods used to generate visualisations from data.

Standards

None

Automation

None

Attributes

Field	Property	Value
ID_VISUALISATION_METHOD	Type	TEXT
	Description	Unique identifier for the VisualisationMethod
	Standards	None
	Validation	Must be unique
	Automation	Automated by the instantiating framework

VisualisationValue

Description

The **VisualisationValue** table links a **Value** to a **Visualisation**, indicating which **Values** are used in a given **Visualisation**.

Standards

PROV:used

Automation

Entries are created automatically when a **Visualisation** is generated.

Relationships

Field	Property	Value
VALUE	Type	TEXT
	Description	ID in Value table of the Value used in the Visualisation .
	Standards	PROV:Entity
	Validation	Must be an ID of a Value .
	Null	Not null.
VISUALISATION	Automation	Populated automatically when the Visualisation is created.
	Type	TEXT
	Description	ID in Visualisation table of the Visualisation using the Value .
	Standards	PROV:Activity.
	Validation	Must be an ID of a Visualisation .
	Null	Not null.
	Automation	Populated automatically when the Visualisation is created.

Bibliography

- Atkinson, Malcolm, Sandra Gesing, Johan Montagnat, and Ian Taylor. 2017. “Scientific Workflows: Past, Present and Future.” In *Future Generation Computer Systems*, vol. 75. Elsevier.
- Christensen, Erik, Francisco Curbera, Greg Meredith, and Sanjiva Weerawarana. 2001. *Web Services Description Language (WSDL) 1.1*. W3C Note. W3C. <https://www.w3.org/TR/wsdl>.
- Davoudian, Ali, and Mengchi Liu. 2020. “Big Data Systems: A Software Engineering Perspective.” *ACM Computing Surveys (CSUR)* 53 (5): 1–39.
- Edmonds, Bruce, and David Hales. 2003. “Replication, Replication and Replication: Some Hard Lessons from Model Alignment.” *Journal of Artificial Societies and Social Simulation* 6 (4): 11. <https://www.jasss.org/6/4/11.html>.
- Edwards, Peter, Edoardo Pignotti, Chris Mellish, et al. 2014. “Lessons Learnt from the Deployment of a Semantic Virtual Research Environment.” *Journal of Web Semantics* 27: 70–77.
- Fanelli, Daniele. 2018. “Is Science Really Facing a Reproducibility Crisis, and Do We Need It To?” *Proceedings of the National Academy of Sciences of the United States of America* 115 (11): 2628–31. <https://doi.org/10.1073/pnas.1708272114>.
- Gao, Yuanzhao, Xingyuan Chen, and Xuehui Du. 2020. “A Big Data Provenance Model for Data Security Supervision Based on PROV-DM Model.” *IEEE Access* 8: 38742–52.
- Hartig, Olaf, and Jun Zhao. 2010. “Publishing and Consuming Provenance Metadata on the Web of Linked Data.” *Provenance and Annotation of Data and Processes: Third International Provenance and Annotation Workshop, IPAW 2010, Troy, NY, USA, June 15-16, 2010. Revised Selected Papers 3*, 78–90.
- Herschel, Melanie, Ralf Diestelkämper, and Houssem Ben Lahmar. 2017. “A Survey on Provenance: What for? What Form? What From?” *The VLDB Journal* 26: 881–906.
- Jin, Xiongbing, Kevin Robinson, Ju-Sung Lee, J. Gareth Polhill, Calvin Pritchard, and Dawn Parker. 2017. “MIRACLE: A Prototype Cloud-Based Reproducible Data Analysis and Visualization Platform for Outputs of Agent-Based Models.” *Environmental Modelling and Software* 96: 172–80. <https://doi.org/10.1016/j.envsoft.2017.06.010>.
- Martin, David, Mark Burstein, Drew McDermott, et al. 2007. “Bringing Semantics to Web Services with OWL-S.” *World Wide Web* 10 (3): 243–77. <https://doi.org/10.1007/s11280-007-0033-x>.
- Oliveira, Wellington, Daniel De Oliveira, and Vanessa Braganholo. 2018. “Provenance Analytics for Workflow-Based Computational Experiments: A Survey.” *ACM Computing Surveys (CSUR)* 51 (3): 1–25.
- Pignotti, Edoardo, Gary Polhill, and Peter Edwards. 2013. “Using Provenance to Analyse Agent-Based Simulations.” In *Proceedings of the Joint EDBT/ICDT 2013 Workshops, Genoa, Italy, March 18-22, 2013*, edited by Kai-Uwe Sattler, Devis Bianchini, Malu Castellanos, et al. Association for Computing Machinery. <https://doi.org/10.1145/2457317.2457371>.
- Polhill, Gary, Terence Dawson, Dawn Parker, et al. 2014. “Towards Metadata Standards for Sharing Simulation Outputs.” In *Advances in Computational Social Science and Social Simulation: Proceedings of the Social Simulation Conference 2014, Barcelona, Catalunya (Spain), September 15*, edited by Francisco J. Miguel, Frédéric Amblard, Joan A. Barceló, and Marco Madella. Autònoma University of Barcelona. <https://core.ac.uk/download/pdf/78523531.pdf>.

- Polhill, Gary, Lorenzo Milazzo, Terence Dawson, Alessandro Gimona, and Dawn Parker. 2017. “Lessons Learned Replicating the Analysis of Outputs from a Social Simulation of Biodiversity Incentivisation.” In *Advances in Social Simulation 2015*, edited by Wander Jager, Rineke Verbrugge, Andreas Flache, Gert de Roo, Lex Hoogduin, and Charlotte Hemelrijk, vol. 528. Advances in Intelligent Systems and Computing. Springer. https://doi.org/10.1007/978-3-319-47253-9_32.
- Pritchard, Nicholas J, and Andreas Wicenec. 2025. “Formal Definition and Implementation of Reproducibility Tenets for Computational Workflows.” *Future Generation Computer Systems* 166: 107684.
- Shrout, Patrick E., and Joseph L. Rodgers. 2018. “Psychology, Science, and Knowledge Construction: Broadening Perspectives from the Replication Crisis.” *Annual Review of Psychology* 69: 487–510. <https://doi.org/10.1146/annurev-psych-122216-011845>.